応用情報技術者試験　テクノロジ系　#12

システム開発技術

用語集まとめ ― 過去問10回分（令和3〜7年度）とシラバスの用語例から



全体像 シラバスの6つの項目を、そのまま6章で読む

1 要件定義
機能要件と非機能要件・ペルソナ・DFD・E-R図・CRUD・決定表・状態遷移表

2 設計
UML・クラスの関係・オブジェクト指向・デザインパターン・MVC・マイクロサービス・モジュールの結合度と強度・品質特性・FTA・レビュー

3 実装・構築
コーディング標準・デバッグ・網羅基準（C0・C1）・同値分割と限界値分析・エラー埋込み法・バグ曲線・PWA

4 統合・テスト
トップダウンとボトムアップ・スタブとドライバ・回帰テスト・カオスエンジニアリング

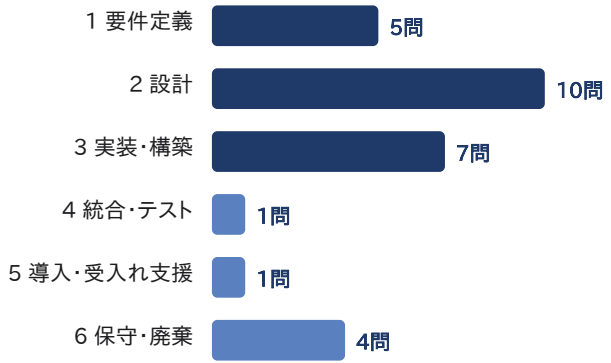
5 導入・受入れ支援
移行方式・ブルーグリーンとカナリア・受入れテスト・教育訓練

6 保守・廃棄
是正・予防・適応・完全化保守・リファクタリング・廃棄

午前での出方	
問題数	毎回 **2〜3問**（問45〜問50あたり）。10回で28問
同じ問題の再出題	**1組**（リバースエンジニアリングで仕様書を作り直す＝完全化保守）。**モジュール結合度は3回**
問題の型	**テストの網羅率の計算**と、**図や手法の名前と説明の入れ替え**が中心
隣の分野	アジャイル・スクラム・知的財産は **13 ソフトウェア開発管理技術** で扱う

この資料の読み方：図や手法は「何を表すか」を一言で言えるようにする。網羅率は流れ図を自分で指でたどって数える。

設計とテストの網羅で6割。図や手法の「入れ替え」が定番



令和3年度春期〜令和7年度秋期の28問を、章ごとに数えた（同じ問題も1問ずつ数えた）

よく出る型	中身	過去問
モジュール結合度	データ結合が最もよい。説明と名前の入れ替え	R4春46・R5春46・R6春46
網羅率	命令網羅（C0）と分岐網羅（C1）の数え方	R3春48・R4春47・R4秋48
UMLの図	アクティビティ図・オブジェクト図の説明	R3秋47・R7秋47
図と表の手法	CRUD・決定表・状態遷移表・DFD・E-R図	R3秋46・R5春47・R3春47
保守のタイプ	問題が起きていない文書の整備＝完全化保守	R4春48・R5秋48
新しい用語	マイクロサービス・PWA・ブルーグリーン・カオスエンジニアリング	R6秋47・R7秋49・R7春46・R7春49

第1章

システム要件定義・ソフトウェア要件定義

機能要件と非機能要件 ／ 要求の集め方とペルソナ ／ DFD ／ E-R図とCRUDマトリクス ／ 決定表 ／ 状態遷移表 ／ プロトタイプとトレーサビリティ



📋「何をするか」が機能要件、「どれだけうまく」が非機能要件

要件	決めること	例
機能要件	システムが行う処理・入出力	受注を登録して在庫を引き当てる
性能要件	レスポンスタイム・スループット	応答3秒以内
可用性要件	稼働率・止めてよい時間	稼働率99.9%
セキュリティ要件	アクセス制御・暗号化・ログ	最小権限で利用者を管理
移行要件	データと業務の移し方	一斉移行でリハーサル2回
運用・保守要件	運用時間・保守の形態	夜間バッチ・遠隔保守

IPAの**非機能要求グレード**は、可用性・性能拡張性・運用保守性・移行性・セキュリティ・システム環境を段階で決める表。発注者と開発者が同じ物差しで合意するために使う。

R6秋 問48：使用性の目標として適切なもの

	目標	実は何の品質か
ア	障害発生から1時間以内に使える	信頼性（回復性）
イ	**使い方を1時間以内に習得できる**	**使用性（習得性）正解**
ウ	要望の改善を1週間以内に完了	保守性（修正性）
エ	使いたい機能が100%ある	機能適合性

要件は**テストで合否が判定できる形**で書く。「使いやすいこと」ではなく「1時間以内に習得できること」のように数値にする（テスト可能性）。

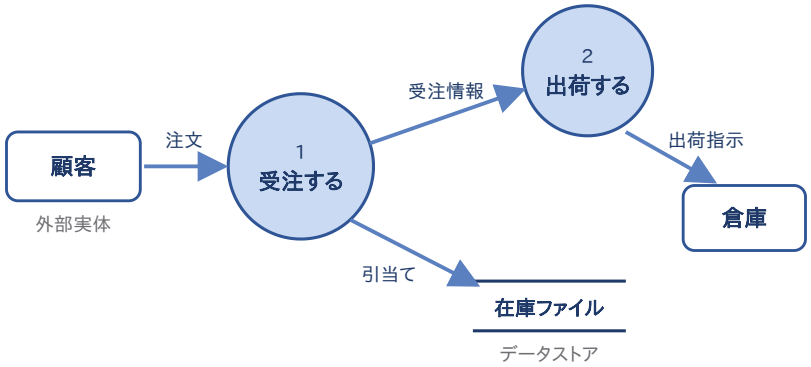
要求は「聞く・配る・見る・集まる」で集め、利用者像はペルソナで描く

方法	中身
ヒアリング	一人ずつ直接聞く。計画を立て、議事録で確認
アンケート	決めた質問を多くの人に配る。広いが浅い
観察法	仕事の様子を見て、口に出ない要求を見つける
JAD	利用者と開発者がワークショップで一緒に決める
MoSCoW分析	Must・Should・Could・Won't で優先順位
5W2H	Why・When・Where・Who・What・How・How much の観点で漏れなく書く

R7秋 問46：利用者を役割ごとに描いた仮想の人物	
選択肢	**本当は**
ペルソナ	**正解**。名前・目的・困りごとを持つ典型的な一人
エピック	1スプリントで作り切れない大きなユーザーストーリー
ステークホルダ	システムに関わる実在の人や組織
プロダクトオーナー	スクラムで優先順位を決める実在の役割

ユーザーストーリーは「〈利用者〉として、〈目的〉のために、〈機能〉がほしい」。大きさは**ストーリーポイント**で相対的に見積もる。**ユースケース**はアクターとシステムのやり取りの手順。

DFDは「データの流れ」だけ。順番や条件は書かない



記号	意味
○ プロセス	データを加工する処理。必ず入力と出力がある
→ データフロー	流れるデータ。名前を書く
＝ データストア	ファイルやデータベース。二本線
□ 外部実体	システムの外の人や組織（源泉と吸収）

いちばん上の**コンテキストダイアグラム**はシステム全体を一つの○にして、外部実体とのやり取りだけを描く。そこから下へ分けていく（**段階的詳細化**）。最下層の○の中身は**ミニスペック**で書く。

データストアどうし・外部実体とデータストアは、間にプロセスを入れずに直接つないではいけない。

E-R図はデータの構造、CRUDマトリクスは「機能×データ」の操作

E-R図：エンティティと関連



CRUDマトリクス：機能が何をするか

機能＼エンティティ	顧客	受注	商品
顧客登録	C		
受注入力	R	C	R
受注取消		D	
価格改定			U

R3秋 問46：CRUDマトリクスの説明はどれか

	説明	本当は
ア	全ての条件と、各条件に対する動作の関係を表形式で	決定表
イ	**各機能がどのエンティティに生成・参照・更新・削除をするか一覧化**	**CRUD（正解）**
ウ	機能と入出力データの関係。データの流れを明確に	DFD
エ	エンティティ・関連・属性の三つでモデル化	E-R図

CRUDマトリクスで「**C（生成）が一つもない列**」は、どこからもデータが作られない＝機能の漏れ。DOA（データ中心アプローチ）では、変わりにくいデータを先に設計する。

決定表は「列ごとに条件を読み、動作を一つ決める」

R5春 問47：3%ずつの値引きを同時に適用								
条件＼列	1	2	3	4	5	6	7	8
上得意客	Y	Y	Y	Y	N	N	N	N
高額取引	Y	Y	N	N	Y	Y	N	N
現金取引	Y	N	Y	N	Y	N	Y	N
値引き	9%	6%	6%	3%	6%	3%	3%	なし

Yの数 × 3% が値引き率。Yが3つ＝9%（列1）、2つ＝6%（列2・3・5）、1つ＝3%（列4・6・7）、0＝なし（列8）。正解は**ウ**。

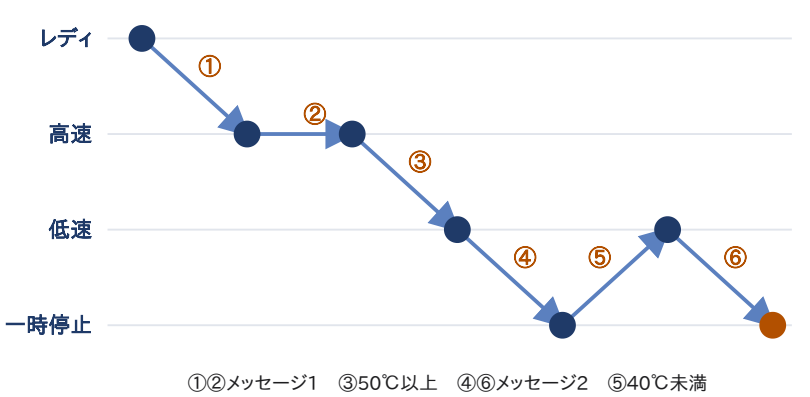
部分	中身
条件記述部	条件の文（上得意客である など）
条件指定部	Y（成り立つ）／N（成り立たない）
動作記述部	行う動作の文（3%値引き など）
動作指定部	X（実行する）／–（しない）

条件が n 個なら組合せは **2^n 通り**（3個で8列）。条件の漏れや重なりが見つけやすく、テストケースもそのまま作れる。

誤りの選択肢は「Yが2つの列を3%にする」「1つの列を値引きなしにする」など、**1列だけずらしてある**。全列を数えて確かめる。

状態遷移表は一つずつたどる。空欄は「変わらない」

R3春 問47：レディから①〜⑥を順に



①②メッセージ1　③50℃以上　④⑥メッセージ2　⑤40℃未満

順	イベント	状態の変化
①	メッセージ1	レディ → 高速運転
②	メッセージ1	高速の欄は**空欄** → 高速のまま
③	温度が50℃以上	減速 → 低速運転
④	メッセージ2	一時停止
⑤	温度が40℃未満	運転再開 → 低速運転
⑥	メッセージ2	**一時停止（エ）**

状態遷移表は「行＝イベント、列＝今の状態」の交わる所に、動作と次の状態を書く。同じ内容を図にしたのが**状態遷移図**、UMLでは**ステートマシン図**。

🔗 作って見せて手戻りを防ぐ。要件はテストまで双方向にたどれるように

試作	中身
モックアップ	画面や帳票の見た目だけ。処理は動かない
プロトタイプ	動く試作品。利用者に触ってもらい仕様の漏れを見つける
水平型	多くの機能を**広く浅く**（画面の流れを確かめる）
垂直型	一部の機能を**深く**（性能や実現性を確かめる）

> 表記法の仲間：BPMN＝業務の流れの標準の書き方（担当ごとのレーン）。SysML＝ハードウェアを含むシステム全体を描くUMLの拡張。

要件の評価	中身
双方向の追跡可能性	要件 → 設計 → テスト、テスト → 要件の両方向にたどれる。漏れと余計なものが分かる
トレーサビリティマトリクス	要件と設計書・テストケースを対応付けた表
外部一貫性・内部一貫性	上位の文書と食い違わない／文書の中で矛盾しない
テスト可能性	テストで合否が判定できる書き方
共同レビュー	発注者（取得者）と開発者（供給者）が一緒に確認
アシュアランスケース	「安全だ」という主張を根拠と論理の筋道で示す文書

第2章

システム方式設計・ソフトウェア設計

UML ／ クラスの関係 ／ オブジェクト指向とSOLID ／ デザインパターン ／ MVC・マイクロサービス ／ 構造化設計と図法 ／ モジュールの結合度・強度 ／ 品質特性 ／ FTA・FMEA ／ レビュー



UMLの図は「形（構造）」と「動き（振る舞い）」に分かれる

構造図（静的）	
クラス図	クラスの属性・操作と、クラスどうしの関係
オブジェクト図	ある時点の**インスタンス**とそのつながり（スナップショット）
コンポーネント図	部品（ソフトウェアの単位）とインタフェース
配置図	ハードウェア（ノード）にどの成果物を置くか
パッケージ図	要素をまとめたグループとその依存
振る舞い図（動的）	
ユースケース図	アクターと、システムが提供する機能
アクティビティ図	処理の流れ。分岐・**並行処理**・同期（フォークとジョイン）
シーケンス図	オブジェクト間のメッセージを**時間の順**に
コミュニケーション図	メッセージを**オブジェクトのつながり**を中心に
ステートマシン図	一つのオブジェクトの状態とイベントによる遷移
タイミング図	状態の変化を時間軸に沿って

R3秋 問47：アクティビティ図はどれか

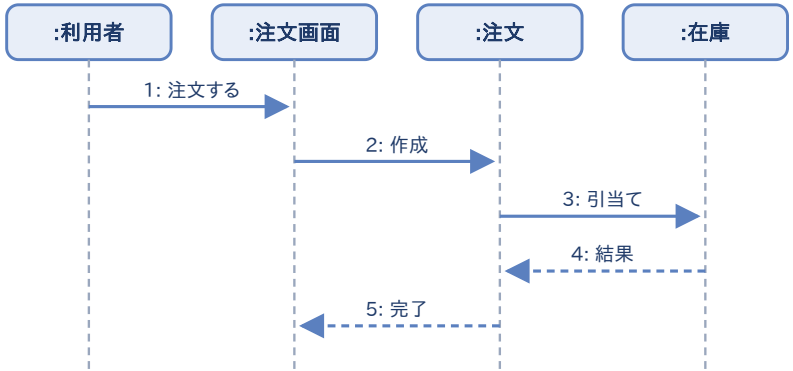
	説明	本当は
ア	**ある振る舞いから次の振る舞いへの制御の流れを表現**	**アクティビティ図**
イ	オブジェクト間の相互作用を時系列で	シーケンス図
ウ	システムが外部に提供する機能と利用者の関係	ユースケース図
エ	オブジェクトの状態がイベントで変わる様子	ステートマシン図

R7秋 問47：インスタンス間の関係を表す図はどれか

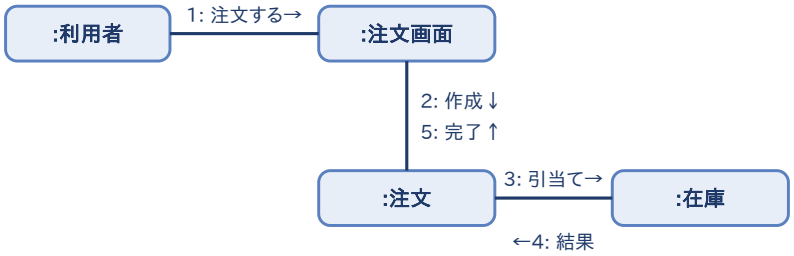
正解は**オブジェクト図**。誤りの選択肢はアクティビティ図（処理の流れ）・コンポーネント図（部品とインタフェース）・ユースケース図（アクターと機能）。**クラス図＝型、オブジェクト図＝ある時点の実物**と覚える。

同じやり取りを「時間の順」で描くか「つながり」で描くか

シーケンス図：縦が時間（上から下へ進む）



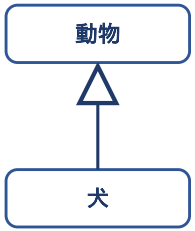
コミュニケーション図：線はつながり、順番は番号で



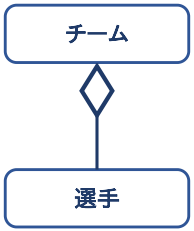
どちらも**相互作用図**の仲間で、同じ内容を描ける。シーケンス図の縦の点線は**生存線（ライフライン）**。破線の矢印は戻りのメッセージ。

ステートマシン図は**一つのオブジェクト**の状態の移り変わり。複数のオブジェクトのやり取りなら、シーケンス図かコミュニケーション図。

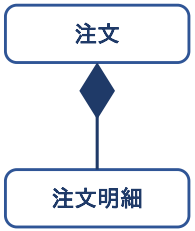
クラス図の線：三角は「一種」、ひし形は「全体と部分」



汎化（is-a）
犬は動物の一種
子が親を継承



集約（ゆるい）
◇側が全体
チームが消えても選手は残る



コンポジション（強い）
◆側が全体
注文と明細は寿命が同じ

関係	記号と意味
関連	実線。多重度（1、0..1、*）を両端に書く
汎化	白抜き三角の実線。親（スーパークラス）を指す
集約	白ひし形。全体と部分（部分は単独でも存在）
コンポジション	黒ひし形。部分は全体と運命を共にする
依存	破線の矢印。一時的に使うだけ
実現	破線＋白抜き三角。インタフェースを実装する

R6春 問45：正しいのは「サブクラスでスーパークラスの操作を**再定義**できる」。誤りは「関連は二つのクラスの間だけ」（三つ以上も可）、「親のインスタンスに操作を頼む」（継承した操作は自分で実行）、「集約では属性と操作を共有する」（共有しない）。

📦 オブジェクト指向：中身を隠し、親から受け継ぎ、同じ呼び方で違う動き

用語	意味
カプセル化	データと操作を一つにまとめ、中身は**メソッド経由でしか触らせない**（情報隠蔽）
継承	スーパークラスの属性・操作をサブクラスが引き継ぐ
多相性（ポリモーフィズム）	同じメッセージを送っても、受け取ったクラスごとに違う動きをする
抽象クラス	インスタンスを作らない親クラス。共通部分だけ決める
インタフェース	操作の名前と形だけ決めた約束。実装は別のクラス
委譲	処理を別のオブジェクトに任せる。継承の代わりに使う

	オーバーライド	オーバーロード
どこで	親と子の間	同じクラスの中
何を	親のメソッドを**同じ名前・同じ引数**で子が**上書き**	**同じ名前で引数の型や数が違う**メソッドを並べる
例	図形.面積() を 円.面積() が書き換える	add(int, int) と add(double, double)

汎化と特化：共通部分を親にくくり出すのが汎化（下から上）、親から具体的な子を作るのが特化（上から下）。**集約と分解**（全体と部分）と取り違えない。

継承を深くしすぎると、親の変更が全部の子に響く。「is-a が成り立たないなら委譲」が設計の目安。

🧩 変更に強いクラスの作り方：SOLIDの原則とデザインパターン

SOLIDの原則	
S	単一責任：クラスを変える理由は一つだけ
O	開放閉鎖：拡張には開き、修正には閉じる（追加で対応）
L	リスコフの置換：子は親の代わりにそのまま使える
I	インタフェース分離：使わない操作に依存させない
D	依存性逆転：具体ではなく抽象（インタフェース）に依存する

> **依存性の注入（DI）**：使う部品を外から渡す。テストのときに偽物（モック）へ差し替えやすい。

GoFのデザインパターン（23種）から、よく出るもの		
分類	**パターン**	**一言**
生成	Singleton	インスタンスを一つだけに限る
	Factory Method	生成をサブクラスに任せる
構造	Adapter	合わないインタフェースを変換する
	Composite	部分と全体を同じように扱う（木構造）
	Facade	複雑な部品群に一つの窓口
振る舞い	Observer	状態の変化を登録先へ通知
	Strategy	アルゴリズムを丸ごと差し替える
	Template Method	処理の骨組みは親、細部は子

アーキテクチャ：役割で分けるMVC、小さく独立させるマイクロサービス

Model
データと業務処理

⇄

Controller
入力を受けて指示

⇄

View
画面に表示

MVCは**表示と処理を分ける**ので、画面だけ変える・処理だけテストする、がしやすい。

方式	中身
レイヤードアーキテクチャ	表示・業務・データの層に分け、上から下へだけ呼ぶ
SOA	業務の機能をサービスとして組み合わせる
サーバーレス	サーバーを意識せず、イベントで関数を実行（FaaS）
イベント駆動	イベントの発生をきっかけに処理が動く

R6秋 問47：マイクロサービスアーキテクチャの利点

	選択肢	本当は
ア	言語やミドルウェアを統一しやすい	サービスごとに選べる。統一しにくい
イ	データの整合性を確保しやすい	データが分散し、難しくなる
ウ	**各サービスの変更がしやすい**	**正解**。独立して変更・配置できる
エ	呼出し回数が減りオーバーヘッド削減	ネットワーク越しの呼出しで増える

サーキットブレーカー：呼び先のサービスが応答しないとき、呼び出しを一時的に遮断して失敗を速く返す。障害が次々に広がる（連鎖障害）のを防ぐ。

ドメイン駆動設計（DDD）：業務の言葉（ユビキタス言語）でモデルを作り、境界づけられたコンテキストごとに分ける。マイクロサービスの分け方の目安になる。

構造化設計：データの流れでモジュールに分け、図で手順を書く

分割手法	分け方
STS分割	データの流れを**源泉（Source）・変換（Transform）・吸収（Sink）**の三つに
TR分割	入力の種類（トランザクション）ごとに処理を分ける
共通機能分割	あちこちで使う処理を共通モジュールにまとめる
ジャクソン法	入出力のデータ構造から、プログラム構造を作る
ワーニエ法	入力データの構造（集合）から、処理の構造を作る

設計の順番は、方式設計（外部設計）→ ソフトウェア設計（内部設計）→ プログラム設計。POA（処理中心）とDOA（データ中心）の違いもここで問われる。

図法	特徴
流れ図	記号と矢印。分岐とループ
NSチャート	四角の入れ子で書く。**矢印がなく**、goto が書けない（構造化を強制）
PAD	木の形。**左から右**へ深く、上から下へ順に読む
HIPO	図式目次＋**入力・処理・出力（IPO）**の総括図と詳細図
構造化チャート	モジュールの呼出し関係を木で

構造化プログラミングの基本は**順次・選択・繰返し**の三つだけで書くこと。どの図法もこの三つを書けるようにできている。

モジュール結合度：受け渡すものが少ないほどよい。データ結合が最良

内容結合	相手の内部を直接参照・変更する
共通結合	グローバルなデータ領域を共有
外部結合	外部宣言した単一データを共有
制御結合	引数で相手の処理の流れを指示（フラグ）
スタンプ結合	構造体を丸ごと渡し、一部だけ使う
データ結合	必要なデータだけを引数で渡す

悪い（強い結合）↑　　↓よい（弱い結合）

10回で3回出た（R4春46・R5春46・R6春46）

問われ方	答え
結合度が最も弱い（よい）もの	データ結合
「引数で処理を切り替えるフラグを渡す」	制御結合
「構造体を渡すが一部しか使わない」	スタンプ結合
「共通領域のデータを複数が参照」	共通結合

覚え方：悪い方から「**内・共・外・制・ス・デ**」。「ないきょう、がいせい、すで」。

外部結合は「単一のデータ項目」、共通結合は「データの集まり（共通領域）」。どちらもグローバルだが、共有の範囲が違う。

💪 モジュール強度：中身のまとまりが強いほどよい。機能的強度が最良

暗合的強度	関係のない機能がたまたま一緒
論理的強度	似た機能をまとめ、引数で一つ選ぶ
時間的強度	同じ時期に実行する処理（初期化など）
手順的強度	決まった順に実行する、関係の薄い機能
連絡的強度	順に実行し、同じデータを使う機能
情報的強度	同じデータを扱う機能を、入口を分けて
機能的強度	一つの機能だけを実行する

悪い（弱い強度）↑　　↓よい（強い強度）

	結合度	強度（凝集度）
見るもの	モジュール**どうし**のつながり	モジュール**の中**のまとまり
よいのは	**弱い**（データ結合）	**強い**（機能的強度）

覚え方：悪い方から「**暗・論・時・手・連・情・機**」。

取り違えやすい組：**論理的**＝「引数で機能を選ぶ」（似た機能の寄せ集め）、**情報的**＝「同じデータを扱う機能を、入口（エントリ）を分けて持つ」。**手順的**と**連絡的**の違いは「同じデータを使うかどうか」。

「結合度は弱く、強度は強く」で、変更の影響が一つのモジュールの中に収まる。

JIS X 25010の製品品質は8特性。副特性から特性を当てる

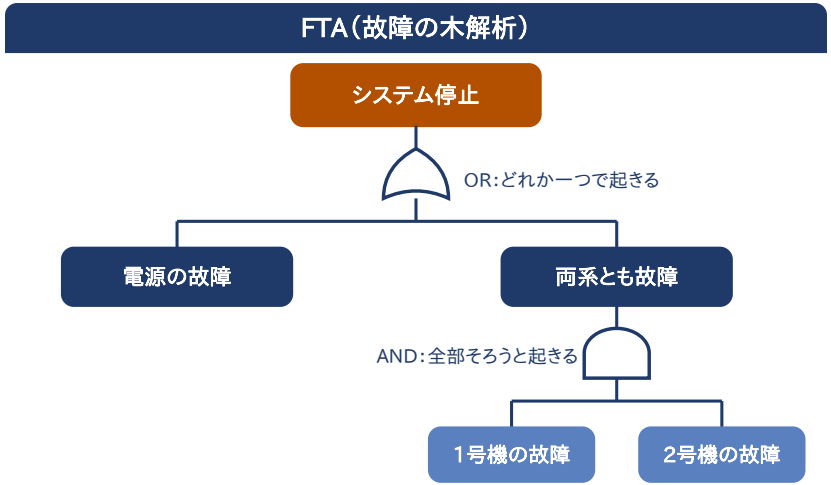
機能適合性	機能完全性・機能正確性・機能適切性
性能効率性	時間効率性・資源効率性・容量満足性
互換性	共存性・相互運用性
使用性	適切度認識性・習得性・運用操作性・ユーザーエラー防止性・ユーザーインタフェース快美性・アクセシビリティ
信頼性	成熟性・可用性・障害許容性・回復性
セキュリティ	機密性・インテグリティ・否認防止性・責任追跡性・真正性
保守性	モジュール性・再利用性・解析性・修正性・試験性
移植性	適応性・設置性・置換性

こう書いてあったら	特性（副特性）
障害が起きても動き続ける	信頼性（障害許容性）
直すべき箇所がすぐ分かる	保守性（解析性）
別の環境へ入れ替えられる	移植性（置換性）
他の製品と資源を分け合って動く	互換性（共存性）
誤操作を防ぐ	使用性（ユーザーエラー防止性）
した操作を後で否定できない	セキュリティ（否認防止性）

利用時の品質は別の5特性：有効性・効率性・満足性・リスク回避性・利用状況網羅性。「使う人から見た品質」。

「可用性」は信頼性の副特性。「保守性」と「修正性」、「信頼性」と「回復性」のように、特性と副特性の階層を混ぜた選択肢に注意。

⚠ FTAは結果から原因へ（上から下）、FMEAは部品から影響へ（下から上）



	FTA	FMEA
出発点	起きてほしくない**結果**（頂上事象）	部品ひとつひとつの**故障モード**
向き	トップダウン	ボトムアップ
道具	AND・ORの論理記号の木	影響・重大さ・発生頻度の表

R4秋 問47：FTAは「障害と、中間的な原因から基本的な原因までを列挙し、**ゲート（論理記号）**で関連付けた樹形図」。誤りの選択肢はFMEA（構成品目の故障モードから影響を評価）・なぜなぜ分析・ODC分析の説明。

設計の安全策の仲間：**フェールセーフ**（故障時は安全側へ止める）、**フェールソフト**（機能を落として続ける）、**フールプルーフ**（誤操作できない作り）、**フォールトトレランス**（冗長化で動き続ける）。

レビューは「誰が進めるか・どれだけ公式か」で名前が変わる

レビュー	進め方
インスペクション	**モデレータ**が進行役。役割・チェックリスト・記録が決まった**最も公式**なやり方
ウォークスルー	**作成者を含む複数人**で、振る舞いを机上でシミュレートしながらたどる（R4秋46）
ラウンドロビン	参加者が**持ち回り**で進行役をする
パスアラウンド	資料を**回覧**して意見を書いてもらう（集まらない）
ピアレビュー	同じ立場の仲間どうしで見る
机上チェック	作成者が一人で見直す

レビューは**成果物の誤りを見つける場**。作成者を評価する場にしない。誤りの直し方の議論は会議の外で。

R4秋 問46：作成者を含めた複数人で、机上でシミュレートするレビュー

正解は**ウォークスルー**。誤りの選択肢のサンドイッチテスト・トップダウンテストは結合テストの進め方、並行シミュレーションはシステム監査の技法。

R7春 問45：デザインレビューの目的

正解は「**設計工程の中で**、設計工程の成果物の問題点を発見する」。出荷の可否の判定・抜取り検査・プロジェクト運営の問題の発見は、どれも別の活動。

入力チェック	確かめること
ニューメリックチェック	数字だけか
範囲（リミット）チェック	上限・下限の間か
チェックディジット	計算した検査用の数字と合うか
照合チェック	マスタに登録された値か

第3章

実装・構築

コーディング標準と静的解析 ／ デバッグの道具 ／ 命令網羅と分岐網羅 ／ 同値分割と限界値分析 ／ エラー埋込み法とバグ曲線 ／ PWAとメディアクエリ



⌨️ 書く前に決まりを決め、書いたら動かさずに調べ、動かして直す

用語	中身
コーディング標準	名前の付け方・字下げ・使ってはいけない書き方を決めた約束。読みやすさをそろえる
静的解析	**実行せずに**ソースコードを調べる（未使用の変数・規約違反など）
動的解析	**実行して**調べる（メモリの使い方・性能）
コードレビュー	人がソースコードを読んで欠陥を探す
リファクタリング	**外から見た動きを変えずに**内部の構造をよくする
ペアプログラミング	二人一組で一台に向かう（13で詳しく）

デバッグの道具	何をするか
ブレークポイント	指定した行で実行を止め、変数の中身を見る
トレーサ	実行した命令の順と結果を記録する
スナップショットダンプ	指定した時点のメモリの内容を出力する
インスペクタ	実行中に変数やデータ構造の中身を表示
アサーション	「ここではこうなっているはず」を書き、外れたら知らせる

> 開発を助ける道具の仲間：**IDE**（統合開発環境）、**バージョン管理**（Git）、**CI**（変更のたびに自動でビルドとテスト）、**ドキュメントジェネレータ**（ソースのコメントから文書を作る）。

ホワイトボックステストの網羅基準は、下にいくほど厳しい

基準	満たす条件
命令網羅（C0）	すべての**命令**を少なくとも1回実行する
判定条件網羅（C1） ＝分岐網羅	すべての**判定**で、真と偽の**両方**の分岐を通る
条件網羅	判定の中の**個々の条件**が真と偽の両方をとる
判定条件／条件網羅	判定条件網羅と条件網羅を両方満たす
複数条件網羅	個々の条件の**真偽の組合せ**を全部試す
経路網羅	入口から出口までの**全経路**を通る

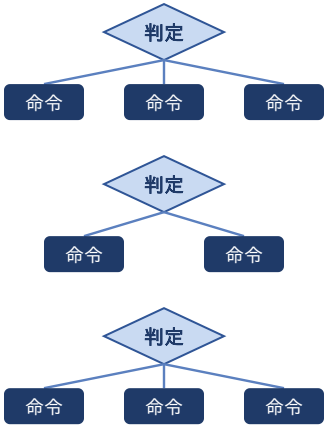
C1を満たせば、必ずC0も満たす。分岐を全部通れば、その先の命令も全部通るから。逆は成り立たない（Noの側に命令がない判定では、Yesだけ通ればC0は満たせる）。

条件網羅は判定条件網羅を含まない。「A or B」で (A真,B偽)(A偽,B真) なら個々の条件は真偽両方をとるが、判定はどちらも真になり、偽の分岐を通らない。

ホワイトボックス＝**中の構造**を見てテストケースを作る。ブラックボックス＝**仕様（入力と出力）**だけを見て作る。

命令網羅の最少ケース数は「いちばん分岐の多い判定の分岐数」

R3春 問48：3分岐 → 2分岐 → 3分岐、命令は全部で8つ



ケース	判定1	判定2	判定3
1	左	左	左
2	中	右	中
3	右	どちらでも	右

1ケースで三つの判定を全部通るので、**3ケース（ア）**で8命令すべてを実行できる。6（3+3）・8（命令数）・18（3×2×3＝経路の組合せ）は誤り。

R4春 問47：判定条件網羅の最少の組

ケース	判定① A＞0 かつ B=0	C	判定② A＞0 かつ C=1
A=1, B=0	Yes → C=1×1	1	Yes
A=1, B=1	No → C=2	2	No

この2ケースで**4つの分岐を全部**通る。正解は**イ**。A=0 のケースは判定①がNoになるだけで、Yesの分岐を通らない。

「全分岐を通れる」選択肢は複数ある。問われているのは**最少**。同じ分岐しか通らない余分なケースが入っていないかを見る。

C0とC1の網羅率は「通った数 ÷ 全部の数」を別々に数える

R4秋 問48：判定5つ・命令5つ、ケースは「全部0」と「全部1」			
判定	全部0	全部1	通った分岐
V=0	Yes → 命令1	No	2/2
(W=0) and (X=0)	Yes → 命令2	No	2/2
(W≠0) and (X≠0)	（通らない）	Yes → 命令3	1/2
(Y=0) and (Z=0)	Yes → 命令4	No	2/2
(Y≠0) and (Z≠0)	（通らない）	Yes → 命令5	1/2

C0 ＝ 5命令 ÷ 5命令 ＝ 100%

C1 ＝ 8分岐 ÷ 10分岐 ＝ 80%

正解は**イ**。「W≠0 and X≠0」の No 側（W と X の片方だけが0）は、どちらのケースでも通らない。

C0がC1より低い組（ウ）はあり得ない。分岐を全部通れば命令も全部通るので、C1が100%ならC0も100%。選択肢をこれで一つ消せる。

📏 ブラックボックス：同じ扱いの範囲から代表を選び、境目は両側を試す

例：年齢が18〜65なら受け付ける



技法	テストデータ
同値分割	各クラスから代表を1つ：10・40・70 の3つ
限界値分析	境界とその隣：17・18・65・66 の4つ

ほかの技法	中身
原因結果グラフ	入力（原因）と出力（結果）の論理関係を図にし、決定表にしてケースを作る
実験計画法（直交表）	因子の組合せを少ないケースで偏りなく試す
ペアワイズ法	二つの因子の値の組合せが必ず1回は出るように
エラー推測	経験から誤りが起きそうな所を狙う
状態遷移テスト	状態遷移表の全遷移を通す

> 無効同値クラスも必ず試す。「受け付けない入力を、ちゃんと受け付けないか」も仕様のうち。

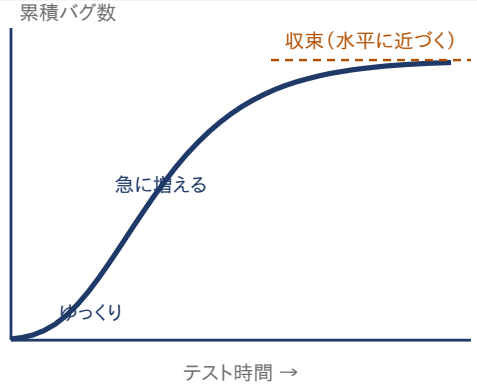
残りのバグ数を見積もる：埋め込んだバグの発見率と、S字のバグ曲線

R7秋 問48：エラー埋込み法

埋め込んだエラー	48個
見つかった埋込みエラー	36個 → 発見率 36÷48=**0.75**
見つかった真のエラー	42個
真のエラーの総数	42÷0.75=**56個**
残っている真のエラー	56−42=**14個（イ）**

56（総数）を答えにしない。問われているのは**残存数**。埋込みエラーは物差しなので、真のエラーには足さない。

R6春 問47：ゴンペルツ曲線



初めはゆっくり、途中で急に増え、終わりは水平に近づく**S字形（ウ）**。水平になった所が「バグが収束した」と判断する根拠。

Webの作り方：一つのHTMLで画面に合わせ、アプリのようにも動かす

R6秋 問50：単一のHTMLで端末の画面に合わせる機能

	選択肢	本当は
ア	User-Agent	ブラウザの種類の情報。サーバが別ページを返すのに使う
イ	WebSocket	接続を保った双方向通信
ウ	マッシュアップ	他社のWeb APIを組み合わせる
エ	**メディアクエリ**	**正解**。CSSで「幅○px以下なら」とスタイルを切り替える

メディアクエリで一つのHTMLのまま画面に合わせる作り方が**レスポンシブWebデザイン**。このサイトもスマートフォン用にこれを使っている。

R7秋 問49：プログレッシブWebアプリ（PWA）

	説明	本当は
ア	サーバ上のプログラムを呼び出し結果を返す	CGI
イ	ページにメタデータを付け機械が処理	セマンティックWeb
ウ	他サービスのWeb APIを組み合わせる	マッシュアップ
エ	**サービスワーカーでオフライン起動・裏で同期し、ネイティブアプリのように**	**PWA（正解）**

PWAのキーワードは**サービスワーカー**。このサイトの `sw.js` もそれで、電波のない所でも教材を開ける。

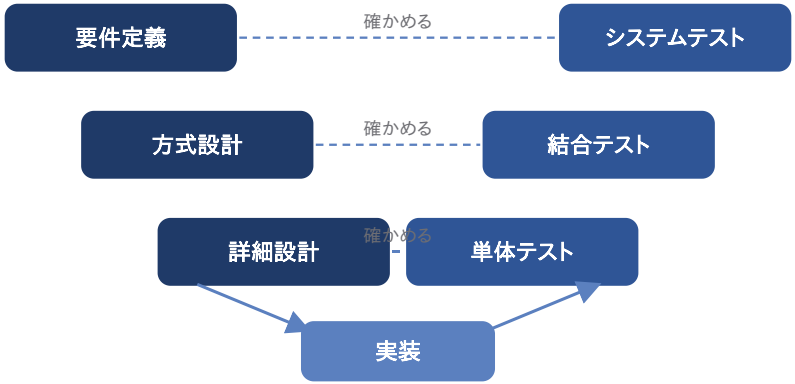
第4章

ソフトウェア結合・システム結合・テスト

V字モデル ／ トップダウンとボトムアップ ／ スタブとドライバ ／ テストの種類 ／ カオスエンジニアリング ／ 検証と妥当性確認



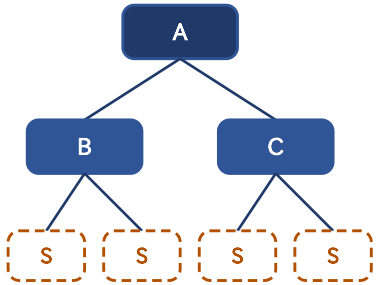
V字モデル：左で決めたことを、右の同じ高さのテストで確かめる



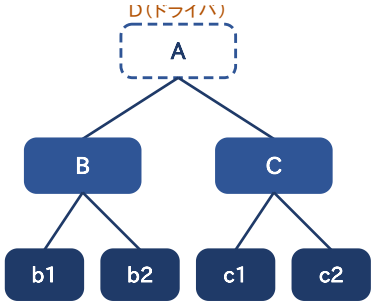
テスト	確かめること
単体テスト	モジュール一つが詳細設計どおりに動くか
結合テスト	モジュールをつないだときのインタフェース
システムテスト	システム全体が要件（機能・性能など）を満たすか。開発者が行う
運用テスト	実際の業務の流れで使えるか。利用者側が行う
受入れテスト	発注者が、契約どおりかを確かめて受け取る

テストは**計画→設計→実行→評価**。テストケースは設計の工程で、右側のテストに合わせて先に作っておける。

上からつなぐなら下の代役（スタブ）、下からつなぐなら上の代役（ドライバ）



トップダウン：下を仮のスタブ（S）で



ボトムアップ：上を仮のドライバ（D）で

進め方	中身	仮のモジュール
トップダウン	上位から順につなぐ。主要な流れを早く確かめられる	**スタブ**（呼ばれる側の代役）
ボトムアップ	下位から順につなぐ。部品の品質を先に固める	**ドライバ**（呼ぶ側の代役）
サンドイッチ	上と下から同時に進め、真ん中で合わせる	両方
ビッグバン	全部一度につなぐ。誤りの場所が分かりにくい	不要

「スタブ＝下、ドライバ＝上」を逆にした選択肢が定番。**スタブは呼ばれる、ドライバは呼ぶ**。

🧪 テストの種類と、本番で小さく壊してみるカオスエンジニアリング

テスト	中身
回帰テスト	修正で、**今まで動いていた所が壊れていないか**（退行テスト・リグレッションテスト）
性能テスト	応答時間・処理件数が要件を満たすか
負荷テスト	想定した最大の負荷をかけても動くか
ストレステスト	想定を超えた負荷で、どう壊れるか
ペネトレーションテスト	攻撃者の立場で実際に侵入を試す（セキュリティテストの一つ）
ユーザビリティテスト	利用者に使ってもらい、使いにくい所を見つける

R7春 問49：カオスエンジニアリングの五つの原則	
1	定常状態における振る舞いの仮説を立てる
2	実世界の事象を多様化させる
3	**本番環境で**実験を行う
4	継続的に実行できるよう実験を自動化する
5	**影響範囲を局所化する**

> 問題は1・2・4を示して、残りの二つを選ばせた。正解は**ウ**「本番環境で」「局所化」。本番の規模や複雑さは開発環境では再現できないので本番で行い、利用者への影響は小さく抑える。

⚖️ 検証は「正しく作ったか」、妥当性確認は「正しいものを作ったか」

	検証（verification）	妥当性確認（validation）
問い	仕様・設計**どおりに**作れているか	利用者の**本当の目的**に合っているか
比べる相手	前の工程の成果物（仕様書）	利用者のニーズ・実際の使われ方
主な手段	レビュー・単体テスト・結合テスト	運用テスト・受入れテスト・試行

仕様書どおりに完璧に作っても、仕様そのものが利用者の目的とずれていれば妥当性確認で落ちる。だから要件定義の段階でプロトタイプやレビューを使う。

テストの終わりの判断	見るもの
バグ曲線	累積バグ数が水平に近づいたか
テスト密度	規模あたりのテスト項目数
バグ密度	規模あたりのバグ数が基準の範囲か
網羅率	C0・C1が目標に届いたか

バグ密度は**低すぎても**注意。テストが不十分で見つけ切れていない可能性がある。

第5章

導入・受入れ支援

移行方式 ／ ブルーグリーン・カナリア・ローリング ／ 受入れテスト ／ 教育訓練と評価



🚚 移行方式は「速さ」と「安全さ」の引換え

一斉移行



ある日に全部切り替える。速いが失敗の影響が大きい

並行移行



結果を比べられて安全。手間と費用は二重

段階移行



業務や機能ごとに順に切り替える

パイロット



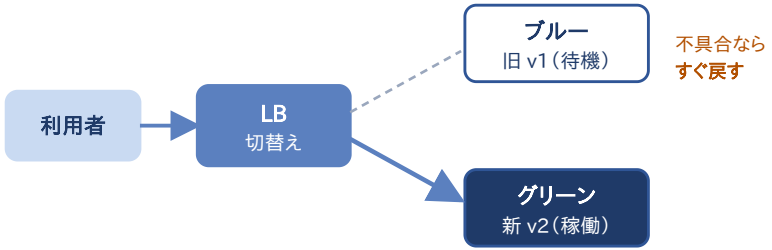
一部の部門で試してから全体へ

方式	向いている場面	リスク
一斉移行	新旧を並べて動かせない、規模が小さい	大
並行移行	誤りが許されない会計・決済など	小（費用大）
段階移行	機能や拠点が分かれている	中
パイロット移行	まず一部で効果と問題を確かめたい	小〜中

移行の前には**移行計画**（手順・判定基準・切り戻しの方法）を作り、**移行リハーサル**で時間と手順を確かめる。データ移行では件数と合計値の突き合わせで漏れを調べる。

🚀 止めずに入れ替える：二つの環境、少しずつ、1台ずつ

R7春 問46：ブルーグリーンデプロイメント



環境を**二つ**用意し、片方を新しい版にしてから、ロードバランサで向き先を**一気に切り替える**。旧環境が残っているので**切り戻しが速い**。正解は**ア**。

手法	中身
ブルーグリーン	環境を二つ。一気に切替え、すぐ切り戻せる
カナリアリリース	新しい版を**一部の利用者だけ**に出し、問題がなければ広げる
ローリングアップデート	複数のサーバを**1台ずつ順に**更新する
ホットスタンバイ	予備機を起動したまま待たせる冗長化（版の入替えではない）
ホットスワップ	電源を入れたまま部品を交換（ハードウェア）

誤りの選択肢はホットスタンバイ・ホットスワップ・ローリングアップデート。「**環境を二つ**」「**切り戻し**」があればブルーグリーン。

🤝 受入れは発注者が自分で確かめる。教育は4段階で効果を測る

用語	中身
受入れテスト	**発注者（利用者側）**が主体で、要件と契約どおりかを確かめる。開発者は支援する
検収	納品物を確かめて受け取ること。支払いのきっかけになる
受入れ基準	何を満たせば合格かを、あらかじめ合意しておく
利用者マニュアル	操作の手順書。受入れのときに一緒に確かめる

受入れテストの**主体は発注者**。「開発者が行うシステムテスト」との入れ替えに注意。

教育訓練の効果：カークパトリックの4段階

レベル1 反応
受講者の満足度（アンケート）

レベル2 学習
知識や技能が身についたか（テスト）

レベル3 行動
仕事のやり方が変わったか（職場での観察）

レベル4 結果
業績や成果にどう表れたか

教育の方法：集合研修・OJT・**e-ラーニング**（WBT＝Webで受ける学習）・ロールプレイング。

第6章

ソフトウェア保守・廃棄

保守の4タイプ ／ 保守性の指標 ／ リファクタリング「継承の分割」／
リバースエンジニアリング ／ 廃棄



保守の4タイプは「問題の前か後か」×「直すのか、よくするのか」

	欠陥を直す（修正）	よくする・合わせる（改良）
問題が起きた後	是正保守 運用中に見つかった問題を直す	適応保守 OSの更新・法改正など環境の変化に合わせる
問題が起きる前	予防保守 表に出る前の潜在的な欠陥を直す	完全化保守 性能や保守性をよくする・文書を整える

緊急保守は、計画外にすぐ行う是正保守。JIS X 0161 の分類。

R4春 問48 ＝ R5秋 問48（同じ問題が2回）

「**問題は発生していない**が、仕様書とソースコードの不整合を解消するため、リバースエンジニアリングで仕様書を作り直す」は**完全化保守（ア）**。

選択肢	当てはまらない理由
是正保守	問題が発生していない
適応保守	環境は変わっていない
予防保守	仕様書の作り直しは欠陥の修正ではない

キーワードで決める：「発生した問題」→是正、「環境の変化」→適応、「潜在的な欠陥」→予防、「保守性・性能の改善」→完全化。

📏 保守性の指標は「直しやすさ」を表す数。信頼性やテストの指標と区別する

R7春 問47：保守性を定量評価する指標		
	式	何の指標か
ア	運用中の不具合件数 ÷ プログラムの規模	信頼性（欠陥密度）
イ	**適正規模の基準を満たすプログラム数 ÷ 総数**	**保守性（正解）**
ウ	テスト済みの分岐数 ÷ 総分岐数	C1網羅率（テストの進み具合）
エ	推定総エラー数 − 摘出エラー数	残存エラー数（信頼性）

大きすぎるプログラムは読みにくく、直したときの影響も調べにくい。**適正な大きさのプログラムの割合**が高いほど保守しやすい。

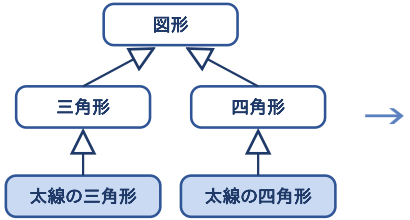
保守しやすくする工夫	中身
サイクロマティック複雑度	分岐の数から求める、プログラムの複雑さ。大きいほど読みにくくテストも多く要る
コメントと設計書の整合	コードを直したら文書も直す
構成管理	どの版に何を直したかを記録し、元に戻せるように
影響分析	直す前に、どこへ影響が及ぶかを調べる

保守の後は**回帰テスト**で、直していない所が壊れていないかを確かめる。

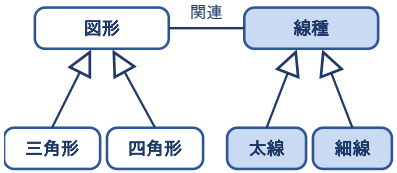
リファクタリング「継承の分割」：二つの観点を別々の継承に分ける

R3秋 問48

変更前



変更後（ウ）



変更前は「形状」と「線種」を一つの継承に混ぜているので、細線を足すと形状の数だけクラスが増える。変更後は**形状を足すなら図形の下に1つ、線種を足すなら線種の下に1つ**で済む。

	選択肢の形	だめな理由
ア	図形の下に組合せの4クラスを並べる	クラスが掛け算で増える
イ	「太線の図形」の下にまた形状ごとのクラス	線種を足すたびに形状の数だけ増える
ウ	**図形の継承と線種の継承を分け、関連でつなぐ**	**正解**
エ	多重継承で太線クラスも継承させる	組合せのクラスが残る

リファクタリングは**外から見た動きを変えない**。機能の追加やバグの修正とは別の作業として行う。

今あるプログラムから設計を起こし、作り直す。最後は安全に廃棄する

ソースコード → 仕様・設計（リバース） → 新しいシステム（フォワード）

用語	中身
リバースエンジニアリング	既存のプログラムを解析して、仕様や設計を取り出す
フォワードエンジニアリング	取り出した仕様をもとに、新しいシステムを作る
リエンジニアリング	リバースとフォワードを続けて行い、作り直す
マイグレーション	今のシステムを新しい環境（OS・クラウド）へ移す

廃棄のときにすること	
廃棄計画	いつ止めるか、データをどうするかを決める
データの移行と保存	法律で保存が必要な記録を残す
データの消去	記憶媒体は**上書き消去・物理的な破壊**で復元できなくする
利用者への通知	停止の日程と代わりの手段を知らせる

> 「ファイルを削除した」「初期化した」だけでは復元できることがある。廃棄の証明書を受け取るのも対策の一つ。

まとめ：この章で迷いやすい組と、見分ける一言

迷いやすい組	見分ける一言	迷いやすい組	見分ける一言
DFD ／ E-R図	データの**流れ** ／ データの**構造**	C0 ／ C1	命令を全部 ／ 分岐の真偽を全部。C1ならC0も
CRUD ／ 決定表	機能×データの操作 ／ 条件の組合せと動作	条件網羅 ／ 判定条件網羅	条件網羅は判定の偽を通らないことがある
シーケンス図 ／ コミュニケーション図	時間の順 ／ つながり	同値分割 ／ 限界値分析	範囲の代表 ／ 境目の両側
クラス図 ／ オブジェクト図	型 ／ ある時点の実物	エラー埋込み法	総数＝真の発見数÷発見率。残存＝総数−発見数
汎化 ／ 集約	一種（is-a）／ 全体と部分	ゴンペルツ曲線	S字。水平で収束
オーバーライド ／ オーバーロード	子が上書き ／ 同名で引数違い	スタブ ／ ドライバ	呼ばれる側（下）の代役 ／ 呼ぶ側（上）の代役
結合度	弱いほどよい。**データ結合**が最良	検証 ／ 妥当性確認	仕様どおりか ／ 目的に合うか
強度	強いほどよい。**機能的強度**が最良	ブルーグリーン ／ カナリア	環境二つで一気に ／ 一部の利用者から
FTA ／ FMEA	結果から原因（上から）／ 部品から影響（下から）	是正 ／ 適応	起きた問題を直す ／ 環境の変化に合わせる
インスペクション ／ ウォークスルー	モデレータが進行 ／ 作成者を含め机上で追う	予防 ／ 完全化	潜在的な欠陥 ／ 保守性・性能をよくする